

[DRIVE2.RU](#)

Automotive social network

[Login](#) [Register](#)

[Buy a car](#) [on Drom Cars](#) [Logbooks](#) [Communities](#) [Magazine](#) [Buyer's Guide](#) [Services and shops](#) [Flea market](#) [The most interesting](#) [Cars for sale](#)

BMW 6 Series (E63)

[All 6 series \(E63\) on DRIVE2](#) [Owner logs](#) [Spare parts in Flea Market](#) [26](#)

- [Cars](#)
- [BMW](#)
- [Episode 6](#)
- [E63](#)
- [BMW 6 Series \(E63\) "Yuzhanka"](#)



[Petrol 4.8 l \(367 hp\), automatic, rear, 2008](#)

Warsaw, Poland

[ROSSKO](#)



[Buy at ROSSKO](#)

Coding with Meaning, Part 6



[Mr.Perfect](#)

March 15, 2016

We're continuing to tinker with the KOMBI, using the knowledge gained in [Part 5](#). In that article, we focused on activating the cruise control's set-speed indicator slider. Since that post was written, some readers have already tried this on their cars. Some succeeded, others didn't. They were unlucky: their E60s were equipped with a stripped-down instrument cluster that physically lacked the slider, so activation was useless. This definitely applies to pre-facelift E60s, most often those produced in Russia. I have no information about the facelift, but I suspect everything is fine there. The ETK catalog doesn't offer any special instrument cluster version without the slider, unlike the E90, which, in the HIGH trim, also offers two instrument cluster options, with and without cruise control. With the E60, however, it's a bit of a guesswork.

UPDATE: As a reminder, everything described in this thread can be easily implemented using [Perfect Toolbox](#). However, to understand the process, you should also read this article.

Another rather popular feature is changing the fuel economy gauge's (instantaneous fuel consumption indicator) functionality to oil temperature. The oil temperature gauge was factory-installed only on certain models of cars considered performance vehicles by the manufacturer. Of those I know, these include the E9x with N54/N55 engines, the E63/E64/E71 with gasoline engines, and, of course, the M-series cars. Even the E60 with the N62 engine was equipped with the same fuel economy gauge.



On the left is the economizer, on the right is the oil temperature.

On American models, the fuel economy unit is measured in the usual miles per gallon. Essentially, the liters are roughly the same.

The oil temperature gauge itself isn't very useful as an indicator of engine warm-up, as it's more accurate to use the coolant temperature. This coolant temperature can be easily seen in the service menu:



BMW Coolant Temperature

The inconvenience is that you have to re-enable the menu every time, and doing this before every trip is tedious. In the pre-facelift E60 instrument cluster, the tachometer featured a special scale indicating the maximum RPM limit.



Warm-up scale

As the engine warmed up, the gauge decreased, indicating to the owner whether the car was safe to drive or not. In reality, driving is always possible; the engine simply cannot be revved beyond the current speed limit. In the facelift, the gauge was removed, leaving the owner with no monitoring tools at all, other than the service menu.

However, many would prefer to see at least an oil temperature indicator instead of the economizer, even though it warms up noticeably more slowly. It's possible to monitor the temperature even when the engine is already warm, thus identifying potential problems in the cooling system. Incidentally, the F-series has reintroduced the temperature indicator, but instead of coolant (as it was on the E39 and others), it now displays oil.

Well, you can still enjoy the temperature gauge. It's unlikely that all E60 and similar car owners will benefit from this, but those with gasoline engines will likely find success. This requires a bit more effort than activating the cruise control, but not by much. From the previous sections on coding, you need to know the following:

- Working with the `codierdaten_lesen` function in Tool32 (see [Part 5](#));
- NETTODAT encoding of arbitrary data (see [Parts 3](#) and [5](#));
- Ability to work in the calculator in "programmer" mode.

In general, it will be necessary to use all the knowledge acquired in the previous parts.

We analyze

So, let's say you own a BMW 5 Series E60 with a gasoline engine and want to see the oil temperature on the dashboard instead of the not-so-useful economizer. No problem! But before you do, make sure you have the latest firmware.

As usual, we'll start by reading the data. We need to read the memory location in the instrument cluster at address **0x3105** using the `codierdaten_lesen` function in Tool32 (the SGBD file is still the same— *KOMB60.PRG*). If you have a standard fuel economy setting in liters per 100 kilometers, the read memory location will look something like this:

4E 2F 0D 00 00 36 00 67 00 98 00 CA 00 00 00 BC 01 78 03 35 05 F1 06 6E

A total of 18h (24) bytes. Those in bold may vary, **but** the rest will most likely be exactly as specified. This memory block stores the settings for the economizer needle (**KVA** , **fuel** consumption **indicator**) .

Before performing any actions with recording new data, be sure to save the response received from the `codierdaten_lesen` function to a text file in a safe place so that it will be possible to restore everything to its original state.

The resulting set of bytes needs to be decrypted:

- Byte 0: Checksum;
- Byte 1: Initial position of the arrow;
- Byte 2: Arrow deceleration factor;
- Bytes 3-12: Input value range;
- Bytes 13-22: Sector division checkpoints;
- Byte 23: Setting bits.

We already know more or less how to calculate the checksum, as well as how to do without it altogether. The rest requires some clarification.

To ensure the needle functions fully as an oil temperature indicator, we only need to change a few things in bytes 3 through 12, plus slightly adjust the setting in the last one, 23. Let's start with the last one, breaking it down bit by bit:

- Bit 0: Arrow function: 0b - economizer, 1b - oil temperature;
- Bit 1-3: Not used, value is always 111b;
- Bit 4-7: Minimum speed for activating readings (relevant for economizer).

The last byte we read has the value **6E** h, which in bit representation means the following:

0110 111 0 b

As a reminder, the bit sequence number is counted from right to left. As we can see, the least significant bit (bit 0) is zero, meaning the needle is operating as an economizer. However, it only begins displaying its current value when the speed reaches **0110** b = 6 kilometers per hour. This speed can be adjusted within four bits, i.e., from 0 to 15 km/h inclusive.

Turn on the temperature display mode:

0110111 **1** b = **6F** h

We set the least significant bit to 1. This is exactly what we did in [the fourth part of](#) the coding to activate the cruise control slider.

Is that all? Of course not. Anyone interested can, in principle, create a NETTODAT file *KVA.MAN* right now with the following content:

B 00 **3105** 00, *0010* , **FF** ,2F, 0D, 00,00,36,00,67,00,98,00, CA, 00,00,00, BC
B 00 **3105** 10, *0008* ,01,78,03,35,05, F1,06, **6F**

The checksum (the first byte of the sequence) has been changed to the obviously incorrect **FF** h, as we already know that with NETTODAT encoding it will be calculated automatically. After encoding, we will undoubtedly obtain some results. Indeed, while driving, the needle will no longer act as a pendulum. And even immediately after turning on the ignition on a cold engine, it will settle near the 7 liters per 100 kilometers (7 liters per 100 kilometers) mark, and as the engine warms up, it will approach the 15 liters per 100 kilometers (15 liters per 100 kilometers) mark and will slowly fluctuate there depending on your driving conditions.

The arrow's behavior isn't very obvious, is it? This behavior is due to the fact that we switched the operating mode but didn't specify the correct range of values the arrow should display. This range is represented by bytes 3 through 12, inclusive. Let's extract them from our data in groups of two bytes:

0000 3600 6700 9800 CA00

Why two bytes? The range of values is defined by five points: the minimum, three intermediate points, and the maximum. Each point is written as a 16-bit number, hence the two bytes. The only important point: the numbers are written to memory in low-endian order, meaning the least significant byte comes first, followed by the most significant byte. Therefore, to correctly represent the hexadecimal value of this number, you first need to reverse the bytes and then convert them to decimal if necessary.

Let's translate each number in order:

low-ending = hex-sort = decimal
0000 = 0000h = 0
3600 = 0036h = 54
6700 = 0067h = 103
9800 = 0098h = 152
CA00 = 00CAh = 202

So the range of values looks like this: 0, 54, 103, 152, 202. Does this ring a bell? I think the economizer showed fuel consumption from 0 to 20 liters... If we divide these values by 10, we get these numbers: 0, 5.4, 10.3, 15.2, 20.2. Those are liters!

But liters aren't enough; we need a temperature range. Since I have an E63 with this exact temperature range, the easiest way is to look at how it's written. Here's what I have written (the entire sequence obtained via **codierdaten_lesen**):

0000 : 2D 20 0D **76 00 94 00 A8 00 BC 00 DA 00 00 00 15**
0010 : 02 78 03 DC 04 F1 06 *6F*

In the last byte, as you can see, the flag (temperature mode bit) is set, i.e. equal to one, so **6F** h. This is logical, but we're more interested in the range itself (highlighted in bold). Let's break it down in order:

low-ending = hex-sort = decimal
7600 = 0076h = 118
9400 = 0094h = 148
A800 = 00A8h = 168
BC00 = 00BCh = 188
DA00 = 00DAh = 218

Meanwhile, the instrument panel scale is marked from 70 to 170 degrees. An attentive reader will immediately recognize a simple pattern: if you subtract 48 (decimal) from each point, we get the following values: 70, 100, 120, 140, 170. Exactly our temperature range! This magical number 48 actually lets us know that we can display oil temperature from -48 to, God forgive me, 65,487 degrees Celsius. Why so many? Because the maximum value of a positive 16-bit number is 65,535, from which we subtract 48 and get this hellish temperature. Of course, in reality, such values won't exist; most likely, there's another limit on the maximum value. The bit depth of the point values is simply standardized for all the needles.

So, has the box been cracked open? Not quite. Almost there, but some things need to be clarified, and not just by the range alone. Note that in the case of fuel consumption, the range point values increase evenly, plus or minus:

0, 54 (**+54**), 103 (**+49**), 152 (**+49**), 202 (**+50**)

There's some variation, but it increases around 51 points. Now let's look at the temperature range:

118, 148 (**+30**), 168 (**+20**), 188 (**+20**), 218 (**+30**)

Either +20 or +30 degrees from the previous point. Or, more accurately, from the average of 120, our points are offset by 20 degrees in both directions, and the outer points by another 30 degrees:

118 (70 °C, -30), 148 (100 °C, -20), 168 (120 °C), 188 (140 °C, +20), 218 (170 °C, +30)

This spread is completely nonlinear, meaning that simply changing the range might give us accurate readings at the extremes, but at intermediate points we'll get nonsense. We need to figure this out.

It's time to look at other data, specifically bytes 13 through 22. These are also dots, five of them, but the numbers are different. Let's take them from the economizer version:

0000 BC01 7803 3505 F106

Let's convert to decimal values:

low-ending = hex-sort = decimal
0000 = 0000h = 0
BC01 = 01BCh = 444 (+444)
7803 = 0378h = 888 (+444)
3505 = 0535h = 1333 (+445)
F106 = 06F1h = 1777 (+444)

Now let's take the data from my dashboard:

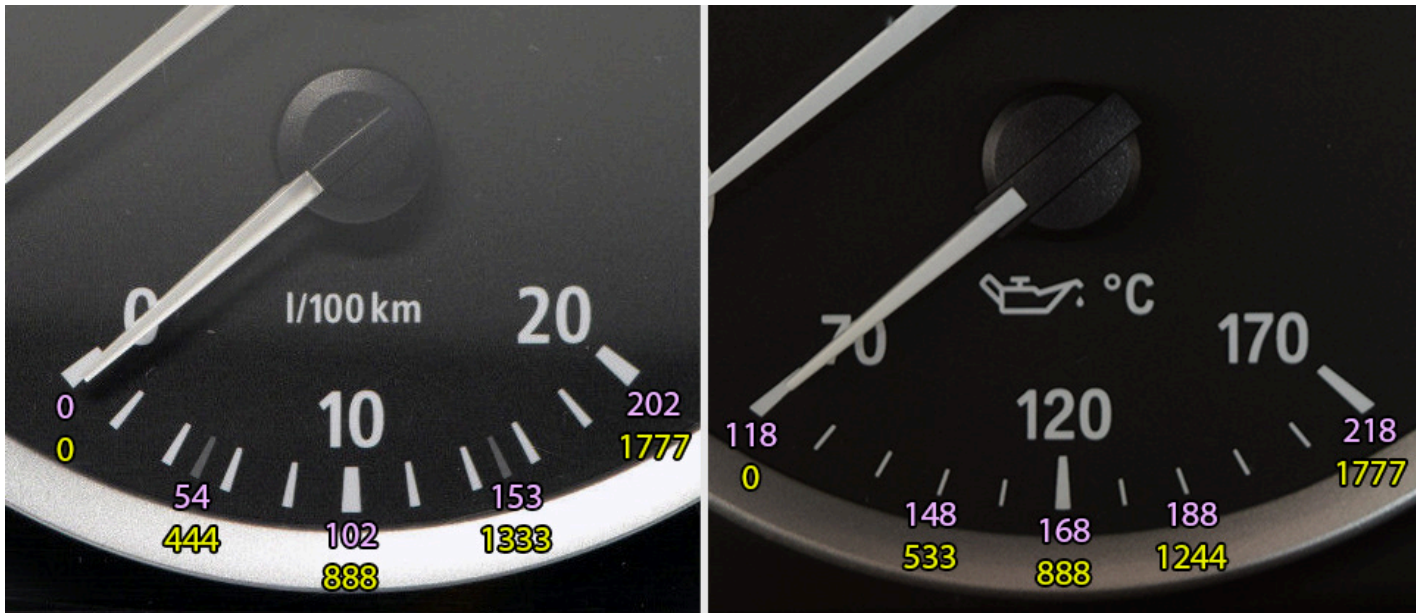
0000 1502 7803 DC04 F106

And we also convert:

low-ending = hex-sort = decimal
0000 = 0000h = 0
1502 = 0215h = 533 (+533)
7803 = 0378h = 888 (+355)
DC04 = 04DCh = 1244 (+356)
F106 = 06F1h = 1777 (+533)

What do these numbers mean? They represent the number of steps the motor (see [Stepper Motor](#)) takes to reach the desired position. This is the position of the hand, expressed in steps relative to the initial (essentially, zero) position.

It's time to visualize all these numbers:



The range point values are shown in pink, and the pointer's control points in motor steps are shown in yellow. Now we have a more or less clear idea of how the pointer works: we have four scale sectors, the boundaries of which we define both by the input value limits (range) and by physical boundaries (control points). We can, for example, specify a small range but a large physical sector to increase the accuracy of readings within a given range of values. Or, conversely, we can specify a large range and fit it within a small physical sector (see example at the end).

In the case of the economizer, the sectors are divided evenly into sections of 444 motor steps. Accordingly, on the physical scale, these points are 0, 5, 10, 15, and 20 liters per 100 km. However, the actual range fluctuates slightly. The first sector is from 0 to 5.4 liters, the second from 5.4 to 10.3, the third from 10.3 to 15.2, and the last from 15.2 to 20.2. Simply put, when you see the needle exactly between 4 and 6 liters (pre-facelift) or 5 liters (facelift), this actually corresponds to 5.4 liters of fuel consumption. In other words, consumption is slightly understated.

Temperature is more strictly represented: the entire 100-degree display range (70 to 170) is divided into four sectors measuring 30, 20, 20, and then 30 degrees again (temperatures, not angles). The sectors are also physically divided in the same ratio: deltas of 355 and 533 steps correspond to 20 and 30 percent of the full scale of 1,777 steps. In other words, there's no trickery in the temperature display; everything is displayed accurately.

Adjusting the economizer

I can't say with complete certainty that the economizer range point values represent fuel consumption in liters per 100 km multiplied by 10, but it's very close. You can try adjusting the points to match the physical representation, specifically, setting the following points: 0, 5, 10, 15, and 20 liters. Why these points? Because all four sectors are exactly the same, unlike the ranges specified for them.

Multiplying by 10, we get: 0, 50, 100, 150 and 200. All that remains is to transform:

decimal = hex-kind = low-ending

0 = 0000h = 0000

50 = 0032h = 3200

100 = 0064h = 6400

150 = 0096h = 9600

200 = 00C8h = C800

The range, therefore, now looks like this: **0000 3200 6400 9600 C800** . We won't change anything else; we can immediately proceed to the formation and encoding of the NETTODAT file *KVA.MAN* :

B 00310500,0010, FF, 2F ,0D, **00,00,32,00,64,00,96,00, C8,00** ,00,00, BC

B 00310510,0008,01,78,03,35,05, F1,06,6E

We've changed the highlighted values, leaving the rest unchanged. Well, except perhaps the checksum. Note the next (second) byte after the checksum: deciphered, this byte controls the initial position of the needle. Its value is specific to each instrument cluster and, in this case, is **2F** h. In your case, you should use the original value obtained from the instrument cluster.

After coding, the economizer should begin displaying fuel consumption correctly. I'll be honest, I haven't tested this yet. Those interested can change the points, go to Test 4 on the instrument panel, and compare the readings with the needle position. There are other adjustments besides this one, but that's beyond the scope of this article. It's not guaranteed that this adjustment will improve the accuracy of the readings, so save your data before experimenting and revert back if necessary.

Converting the economizer to oil temperature

We can set any temperature range, basically. All that remains is to correctly specify the intermediate points so they fit into our sector division. If you initially have a standard fuel economy unit with markings from 0 to 20 liters per 100 km, then in all likelihood the entire range is also divided into identical sectors, for example, like those above:

0000 BC01 7803 3505 F106

There are 5 points, 444 steps between them. Therefore, we must also divide the required temperature range into equal parts. Next, we'll need to determine which economizer you have: a pre-facelift model with sparse markings (see the first photo in the article, left side) or a facelift model with fine markings:



Sorry for the quality of the photo, they didn't have normal cameras during [the Severyanka](#) era .

The following range options are suitable for the small restyling markings:

** Option 0-200 degrees **

The range seems odd at first glance, but on second glance, you can figure out that it's easy to track on a scale from 0 to 20 liters by simply multiplying the needle's reading by 10. So, in degrees, we have the following range points: 0, 50, 100, 150, 200 degrees. Adding the magic 48, we get: 48, 98, 148, 198, 248. All that remains is to translate this into a form understandable for the dashboard:

decimal = hex = little-endian

48 = 0030h = 3000

98 = 0062h = 6200

148 = 0094h = 9400

198 = 00C6h = C600

248 = 00F8h = F800

Or in one line: **3000 6200 9400 C600 F800**

NETTODAT for encoding:

B 00310500,0010, FF, 2F, 0D, **30,00,62,00,94,00, C6,00, F8,00** ,00,00, BC

B 00310510,0008,01,78,03,35,05, F1,06, 6F

Don't forget to leave the value of the second byte, which is responsible for the initial position of the arrow, unchanged.

That's it, you're ready to test. No need to warm up the engine; you can check it using Tool32. However, it won't help you with the 0-200 range:

EDIABAS ToolSet 32

File Job Test Configuration Trace Windows Help

EDIABAS ToolSet 32

Select Job: KOMB60

Jobs

- steuern_leuchten_gruen
- steuern_leuchten_orange
- steuern_leuchten_rot
- steuern_leuchte_aus
- steuern_oeltemp
- steuern_oeltemp_aus

Arguments

Data: ☐

100

Clear Argument wizard

Results

Argumente: 1

job_status
oeltemp_min

Job-Info

name : STEUER
comment : Oeltemp
KWP200

argument : BYTE0
type : int

Results

```
apiJob("KOMB60","steuern_oeltemp","100","")
```

Satz : 0

OBJECT	=	komb60
SAETZE	=	1
JOBNAME	=	steuern_oeltemp
VARIANTE	=	KOMB60
JOBSTATUS	=	
UBATTCURRENT	=	-1
UBATTHISTORY	=	-1
IGNITIONCURRENT	=	-1
IGNITIONHISTORY	=	-1

Satz : 1

JOB_STATUS	=	ERROR_BEREICH_ARGUMENTE
OELTEMP_MIN	=	0 00 00 ..
OELTEMP_MAX	=	200 00 C8 .E
TEL_ANTWORT	=	31 Bytes
0000	:	9B F1 60 62 31 05 AB 20 00 30 00 6C 00 94 00 BC ?n`b1.« .0.1.?.?
0010	:	00 F8 00 00 00 1E 02 87 03 F1 04 0F 07 6F 64 .0.....? .n...od

0x9315	GWS2_FEHLER_CAS_ODER_F
0x9316	GWS2_FEHLER_EEPROM
0x9317	EEPROM: Fehler Codierc
0x9319	HEBELGEBER1

ERROR_BEREICH_ARGUMENTE

I ran the **steuern_oeltemp** function with parameter 100 from the SGBD file *KOMB60.PRG* (this is the same *KOMB60.PRG* we used to extract the coding data earlier and now). This function sets the former economizer needle to the specified value. To reset the readings to the current values, you need to run the **steuern_oeltemp_aus** function or cycle the ignition.

*NB: Functions like **steuern_XXXX** do not take sectoral divisions into account. Only the minimum and maximum extreme points are considered; the rest of the arrow's range is linear. This isn't a major concern if the range point values correspond to this division, but there are cases where they diverge, which can cause problems.*

In this case, I entered 100 degrees, expecting the needle to hover at 5:30. However, that didn't happen: the function returned an error indicating the value was out of range. However, it also states that the range is from 0 to 200 degrees, meaning we definitely coded everything correctly. Nevertheless, if you turn on the ignition or start the car, the needle will immediately show the current oil temperature. However, it's a bit difficult to use as a warm-up indicator:



The water has already warmed up to 69 degrees and is long ready to drive, but the oil temperature has barely crossed the second mark, which, in the 0-200 degree range, means something around 42-45 degrees. In other words, the oil warms up noticeably more slowly. In my case, to warm it up to the usual 92, I need to drive almost 20 kilometers. Or I'd have to idle for half an hour, which is known to be harmful.

Another point is that there's no other way to check the **oil** temperature except by the gauge. Or maybe I just couldn't find any on the N62 (I didn't check on other engines). Diagnostic programs simply don't have such indicators. Neither the KOMBI nor the engine block diagnostics had such functions; they only showed coolant temperature. It's absurd, but that's how it is. Even [the simplest scanner](#) that claims to show engine oil temperature (automatic transmission diagnostics) actually lies and shows coolant temperature, and this is even visible in the photos in the post about it.

** Option 80-120 degrees **

In principle, this is a good option, especially since the number 10 on the economizer will correspond to one hundred degrees, and 20 to 120.

So, the range is 80-120, the sectors are identical, which means the temperature points are: *80, 90, 100, 110, 120*. We add the magic 48 and calculate:

decimal = hex = little-endian

128 = 0080h = 8000

138 = 008Ah = 8A00

148 = 0094h = 9400

158 = 009Eh = 9E00

168 = 00A8h = A800

In one line: **8000 8A00 9400 9E00 A800**

NETTODAT:

B 00310500,0010, FF, 2F ,0D, **80,00,8A, 00,94,00,9E, 00, A8,00** ,00,00, BC

B 00310510,0008,01,78,03,35,05, F1,06, 6F

I'll leave a reminder about saving the value of the initial position byte for the third time, alright. But I won't do it again.

We're already testing 90 degrees with the same **steuern_oiltemp** :

EDIABAS Toolset 32

File Job Test Configuration Trace Windows Help

EDIABAS ToolSet 32

Select Job: KOMB60

Jobs

steuern_leuchten_gruen
steuern_leuchten_orange
steuern_leuchten_rot
steuern_leuchte_aus
steuern_oeltemp
steuern_oeltemp_aus

Arguments

Data: ☐

90

Clear Argument wizard

Results

Argumente: 1

job_status
oeltemp_min

Job-Info

name : STEUER
comment : Oeltem
KWP200

argument : BYTE0
type : int

Results

```

apiJob("KOMB60","steuern_oeltemp","90","")

Satz : 0
OBJECT                = komb60
SAETZE                 = 1
JOBNAME                = steuern_oeltemp
VARIANTE               = KOMB60
JOBSTATUS              =
UBATTCURRENT           = -1
UBATTHISTORY           = -1
IGNITIONCURRENT        = -1
IGNITIONHISTORY        = -1

Satz : 1
JOB_STATUS             = OKAY
OELTEMP_MIN            = 80          00 50 .P
OELTEMP_MAX            = 120         00 78 .x
_TEL_ANTWORT           = 6 Bytes
0000 : 82 F1 60 70 23 66          ?n`p#f

```

0x9315 GWS2_FEHLER_CAS_ODER_F

0x9316 GWS2_FEHLER_EEPROM

0x9317 EEPROM: Fehler Codierc

0x9319 HEBELGEBER1

Now everything is OKAY

I definitely prefer this range of the function. Here's 90 degrees:



If I had the same markings as the restyled economizer, 90 degrees would correspond to 5 liters per 100 kilometers, 10 liters = 100 degrees, 15 = 110, which is generally convenient. However, given that the engine can get very hot, 120 degrees might be too low. But at least it would be immediately obvious that the crankcase is hot.

** Factory options **

These options are suitable for those with a pre-facelift economizer with markings identical to oil temperature. In this case, a range of 100 degrees is ideal, i.e., the factory 70-170 or the "EM" 50-150. The only caveat is that the originals have a specific breakdown by both range points and sectors (as we already discussed above), so I will provide below both the pure factory ones and those optimized for the economizer, with uniform sector breakdowns.

Let's start with the "civilian" 70-170. This is just my factory version:

B 00310500,0010, FF, 20,0D, **76,00,94,00, A8,00, BC, 00, DA, 00** , 00,00,15
 B 00310510,0008, 02,78,03, DC, 04, F1,06,6F

Here we have points at 70, 100, 120, 140, and 170 degrees, divided into sectors accordingly. And my starting position, as you can see, is not **2F** h, as in the examples above, but **20** h. Yours may be different.

Now here is an adapted version for uniform division of economizer sectors:

B 00310500,0010, FF, 20,0D, **76,00,8F, 00, A8,00, C1,00, DA, 00** , 00,00, BC
 B 00310510,0008, 01,78,03,35,05, F1,06,6F

Here the points are 70, 95, 120, 145 and 170 degrees, which should be more or less suitable for owners of pre-restyling economizers.

Let's move on to the "M" version of 50-150 degrees. Here's the factory version of a real M-ka (E60):

B 00310500,0010, FF, 2A, 0D, **62,00,80,00,94,00, A8,00, C6,00** , 00,00,15
 B 00310510,0008, 02,78,03, DC, 04, F1,06,6F

The range point values are: 98, 128, 148, 168, and 198, i.e., 50, 80, 100, 120, and 150 degrees. Essentially identical to the "civilian" 70-170, just offset by 20 degrees.

The "EMOS" version, adapted for the economizer:

B 00310500,0010, FF, 2A, 0D, **62,00,7B, 00,94,00, AD, 00, C6,00** , 00,00, BC
 B 00310510,0008, 01,78,03,35,05, F1,06,6F

In general, you are free to come up with your own version of the range, especially if it is possible to make new scales.

The DRIVE2 editor is gently hinting that it's time to wrap things up, or else there won't be enough words. Nevertheless, some observations are interesting.

I have some factory calibrations from E70 and E71 models, and they're quite interesting. For example, the oil temperature gauge settings from a European 2009 X6 M:

39 0D 62 00 94 00 A3 00 AD 00 C6 00 00 00 [20 03 6E](#) 03 35 05 E1 06 7F

Temperature points: **50, 100 (!), 115, 120, 150** degrees! The sector breakdown is as follows: 0, 800 (!), 878 (!), 1333, 1761. Here's what it looks like visually:



I'm not implying anything, but BMW believes that 100 and 115 degrees are mere cents, and not worth noting. However, unlike the N62, those interested can still check the actual oil temperature of the S63 engine using the **status_oel** function in *MSD85*. It's also likely possible to check it with standard diagnostic programs, since such a function exists. I predict that after this article, garage technicians will offer a new additional service, something like "Oil Temperature Gauge Adjustment for X5 M / X6 M."

Interestingly, on E6x models, the economizer's sector breakdown is identical down to the last increment, with only the initial needle positions varying. On E7x models, however, slight variability in values is noticeable, indicating additional instrument cluster calibration at the factory.

There are just over 300 characters left, which is too much to fit anything useful into. In the next part, we'll focus on calibrating the hands. It's not complicated, but the knowledge from this section is essential. I'll skip the teasers this time.

[Part Seven](#)

Mileage: 190,093 km

March 15, 2016 Tags: [tuning](#)

[212](#)



Spare parts and tires